Python Operators – Complete Notes

Beginner to Interview Level

Definition: An operator is a symbol or keyword used to perform an operation on one or more values. The values on which an operator works are called operands.

Example:

```
a = 10
b = 5
print(a + b)

Output: 15

Here, + is the operator and a, b are operands.
```

1. Arithmetic Operators

Meaning:

Used to perform mathematical calculations.

Operators:

+ - * / // % **

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor Division	10 // 3	3
%	Modulus / Remainder	10 % 3	1
**	Exponent / Power	10 ** 3	1000

Example:

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Important:

/ returns normal division (usually float). // returns floor division. % returns the remainder. ** performs power/exponentiation.

2. Comparison / Relational Operators

Meaning:

Used to compare two values. The result is normally True or False.

Operator	Meaning	Example
==	Equal to	10 == 10
!=	Not equal to	10 != 5
>	Greater than	10 > 5
<	Less than	5 < 10
>=	Greater than or equal	10 >= 10
<=	Less than or equal	5 <= 10

Example:

```
a = 10
b = 5

print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Important:

= means assignment, while == means comparison.

3. Assignment Operators

Meaning:

Used to assign a value or update the value of a variable.

Operator	Example	Equivalent
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
//=	x //= 5	x = x // 5
%=	x %= 5	x = x % 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
|=	x |= 5	x = x | 5
^=	x ^= 5	x = x ^ 5

Operator	Example	Equivalent
>>=	x >>= 2	x = x >> 2
<<=	x <<= 2	x = x << 2

Example:

```
x = 10
x += 5
print(x)

Output: 15
```

4. Logical Operators

Meaning:

Used to combine or reverse conditions.

Operators:

and, or, not

and:

```
Returns True only when both conditions are True.

age = 25
print(age >= 18 and age <= 60)
Output: True
```

or:

```
Returns True when at least one condition is True.

age = 15
print(age < 18 or age > 60)
Output: True
```

not:

```
Reverses a Boolean result.

x = True
print(not x)
Output: False
```

5. Identity Operators

Meaning:

Check whether two names refer to the same object.

Operators:

is, is not

is:

```
a = [1, 2, 3]
b = a
print(a is b)
Output: True
```

is not:

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is not b)
Output: True
```

Important:

Use == for value equality. Use is for object identity. A common use of is is checking against None, for example: if value is None:

6. Membership Operators

Meaning:

Check whether a value exists in a sequence or collection.

Operators:

in, not in

Example:

```
numbers = [10, 20, 30, 40]

print(20 in numbers)
print(50 in numbers)

Output:
True
False
```

String Example:

```
text = "Python"
print("Py" in text)
Output: True
```

7. Bitwise Operators

Meaning:

Work on the binary representation of integers.

Operators:

& | ^ ~ << >>

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

Example:

```
a = 5
b = 3

5 in binary = 101
3 in binary = 011

print(a & b) # 1
print(a | b) # 7
print(a ^ b) # 6
```

NOT:

For integers, ~n = -(n + 1). Therefore, ~5 = -6.

Shift:

```
5 << 1 = 10
20 >> 1 = 10
```

8. Conditional / Ternary Expression

Meaning:

A one-line form of if-else.

Syntax:

value_if_true if condition else value_if_false

Example:

```
age = 20
result = "Adult" if age >= 18 else "Minor"
print(result)
Output: Adult
```

Example 2:

```
num = 10
result = "Even" if num % 2 == 0 else "Odd"
print(result)
Output: Even
```

9. Operators with Strings and Lists

Strings:

```
The + operator concatenates strings.

"Akshay" + " " + "Sadare" → "Akshay Sadare"

The * operator repeats a string.

"Python " * 3 → "Python Python Python"

in checks membership.
```

Lists:

```
The + operator combines lists.

[1, 2] + [3, 4] → [1, 2, 3, 4]

The * operator repeats a list.
```

```
[1, 2] * 3 → [1, 2, 1, 2, 1, 2]
```

10. Unary Operators

Meaning:

An operator that works on one operand.

Examples:

```
x = 10
+x → 10
-x → -10
~x → -11
```

11. Walrus Operator (:=)

Meaning:

Assignment expression operator. It assigns a value as part of an expression.

Example:

```
if (n := 10) > 5:
print(n)

Output: 10
```

Note:

This is an advanced feature. Beginners should learn normal assignment first.

12. Operator Precedence

Meaning:

When an expression contains multiple operators, Python follows a precedence order.

Example:

```
10 + 5 * 2

First: 5 * 2 = 10
Then: 10 + 10 = 20
```

Important Order:

```
A useful simplified order is:

1. () Parentheses
2. ** Exponentiation
3. +x, -x, ~x Unary operators
4. *, /, //, %
5. +, -
6. <<, >>
7. &
8. ^
9. |
10. Comparisons: <, <=, >, >=, ==, !=
11. not
12. and
```

```
13. or
```

Parentheses:

```
Use parentheses when you want to make the intended order clear.

(10 + 5) * 2 = 30
```

13. Complete Operator Cheat Sheet

Arithmetic:

+ - * / // % **

Comparison:

== != > < >= <=

Assignment:

= += -= *= /= //= %= **= &= |= ^= >>= <<=

Logical:

and or not

Identity:

is is not

Membership:

in not in

Bitwise:

& | ^ ~ << >>

Conditional:

x if condition else y

Assignment Expression:

:=

14. Interview Important Questions

Q1. Difference between / and //?:

/ performs normal division; // performs floor division.

Q2. Difference between = and ==?:

= assigns a value; == compares values.

Q3. Difference between == and is?:

== compares values; is checks object identity.

Q4. What does % do?:

It returns the remainder.

Q5. What does ** do?:

It performs exponentiation.

Q6. What does and require?:

Both conditions must be True.

Q7. What does or require?:

At least one condition must be True.

Q8. What does not do?:

It reverses a Boolean result.

Q9. What does in do?:

It checks membership in a sequence or collection.

Q10. What is the ternary operator in Python?:

Python uses the conditional expression: value_if_true if condition else value_if_false.

15. Quick Revision

Remember:

```
Arithmetic → Calculate
Comparison → Compare
Assignment → Assign/Update
Logical → Combine conditions
Membership → Check inside
Identity → Check same object
Bitwise → Work with binary bits
Ternary → One-line if-else
```

Practice:

```
Try writing programs for:
1. Check even/odd using %
2. Find the remainder of two numbers
3. Find the square/cube using **
4. Compare two numbers
5. Check age using and/or
6. Check whether an item exists in a list using in
7. Use == and is with lists
8. Perform bitwise operations on two integers
9. Use a ternary expression for pass/fail
```

End of Notes — Python Operators